

# The Complete Docker Guide: From Zero to Swarm

A step-by-step journey through Docker — each step builds on the last and gets progressively more advanced. Every command is commented so you understand *what* it does and *why*.

---

## STEP 1: Docker Theory — What Even Is Docker?

### The Problem Docker Solves

Before Docker, developers dealt with the classic complaint: "**It works on my machine!**" A Python app might work perfectly on your laptop but break on your colleague's machine or on the production server, because of differences in:

- Operating system version
- Installed library versions
- Environment variables
- System configuration

### Virtual Machines vs Containers

**Virtual Machines (VMs)** solve this by virtualizing an *entire computer*, including its own OS kernel. Each VM is heavy (gigabytes), slow to start (minutes), and wastes resources because every VM carries a full OS.

**Containers** solve the same problem but much more efficiently. A container:

- Shares the **host machine's OS kernel** (no need to boot a separate OS)
- Packages only the **application + its dependencies** (libraries, binaries, config files)
- Starts in **seconds** (sometimes milliseconds)
- Is **lightweight** (megabytes, not gigabytes)

Think of it like this:

- A **VM** is like building a separate house (with its own foundation, plumbing, electricity) for every tenant.
- A **container** is like an apartment in a shared building — each apartment (container) is isolated and has its own stuff, but they all share the building's foundation and utilities (the host OS kernel).

## Key Docker Concepts (Vocabulary You Need First)

| Term                 | Meaning                                                                                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Image</b>         | A read-only blueprint/template for a container. Contains the app code, runtime, libraries, and dependencies. Think of it as a "recipe" or "class" in OOP terms. |
| <b>Container</b>     | A running (or stopped) <i>instance</i> of an image. Think of it as an "object" created from the image "class".                                                  |
| <b>Dockerfile</b>    | A text file with instructions to build a Docker image.                                                                                                          |
| <b>Docker Engine</b> | The background service (daemon) that builds and runs containers.                                                                                                |
| <b>Docker Hub</b>    | A public registry (like GitHub, but for images) where you can pull/push images.                                                                                 |
| <b>Registry</b>      | Any server that stores Docker images (Docker Hub is the default public one; you can also run private registries).                                               |
| <b>Volume</b>        | A mechanism to persist data generated by containers, outside the container's writable layer.                                                                    |
| <b>Network</b>       | A mechanism that lets containers talk to each other and to the outside world.                                                                                   |

## How Docker Achieves Isolation (Brief Theory)

Docker uses Linux kernel features:

- **Namespaces** — isolate what a process can see (its own PID list, network interfaces, mount points, hostname, etc.) so a container thinks it's the only thing running.
- **cgroups (control groups)** — limit what a process can *use* (CPU, memory, disk I/O), preventing one container from hogging all host resources.
- **Union File Systems (like OverlayFS)** — allow images to be built in stacked, reusable **layers**. Each instruction in a Dockerfile creates a new layer, and layers are cached and shared between images, saving disk space and build time.

---

## STEP 2: Installing Docker & Your First Commands

### Installation (brief overview)

- **Linux:** Install via your package manager or the convenience script:

```
bash
```

```
# Downloads and runs Docker's official install script
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

- **Windows/Mac:** Install **Docker Desktop** from [docker.com](https://docker.com) (it includes a lightweight Linux VM under the hood, since Docker fundamentally needs a Linux kernel).

Verify installation:

```
bash
```

```
# Prints the installed Docker version - confirms Docker CLI is installed correctly
docker --version

# Runs a tiny test container to confirm Docker Engine (the daemon) is working
docker run hello-world
```

## Core Commands: Working with Images

```
bash
```

```
# Downloads (pulls) an image from Docker Hub (the default registry)
# Format: docker pull <image-name>:<tag> (tag defaults to "latest" if omitted)
docker pull nginx:latest

# Lists all Docker images currently stored on your machine
docker images
# or the newer syntax:
docker image ls

# Removes a specific image from your local machine
# You must specify the image ID or name:tag
docker rmi nginx:latest

# Removes ALL unused (dangling) images to free up disk space
docker image prune
```

## Core Commands: Working with Containers

bash

```
# Creates AND starts a new container from an image
# -d = detached mode (runs in background, doesn't block your terminal)
# --name = gives the container a friendly name instead of a random one
# -p 8080:80 = maps host port 8080 to container port 80 (host:container)
docker run -d --name my-nginx -p 8080:80 nginx:latest

# Lists all RUNNING containers
docker ps

# Lists ALL containers, including stopped ones
docker ps -a

# Stops a running container gracefully (sends SIGTERM, waits, then SIGKILL if n
docker stop my-nginx

# Starts a previously stopped container (reuses same container, doesn't create
docker start my-nginx

# Restarts a container (stop + start combined)
docker restart my-nginx

# Removes a stopped container permanently (frees up the name and container ID)
docker rm my-nginx

# Force-removes a RUNNING container (stops it first, then removes) - use with c
docker rm -f my-nginx

# Shows real-time logs from a container (great for debugging)
# -f = follow (like `tail -f`), keeps streaming new logs
docker logs -f my-nginx

# Opens an interactive shell INSIDE a running container
# -i = interactive (keep STDIN open) -t = allocate a pseudo-terminal (nice for
docker exec -it my-nginx bash

# Shows live resource usage (CPU, memory, network) for running containers
docker stats

# Shows detailed metadata about a container (IP address, mounts, env vars, etc.
docker inspect my-nginx
```

**Try it yourself:** Run `docker run -d --name my-nginx -p 8080:80 nginx`, then open `http://localhost:8080` in your browser — you'll see the Nginx welcome page, proving the container is serving traffic.

---

## STEP 3: Building Your Own Docker Images (The Dockerfile)

A **Dockerfile** is a script of instructions that tells Docker how to build an image, layer by layer.

### Example: Dockerizing a Simple Node.js App

Project structure:

```
my-app/  
├─ Dockerfile  
├─ package.json  
└─ server.js
```

**Dockerfile**:

dockerfile

```
# FROM specifies the BASE IMAGE to build on top of.
# "node:18-alpine" = Node.js v18 on Alpine Linux (a minimal, small Linux distro)
FROM node:18-alpine

# WORKDIR sets the working directory INSIDE the container for all subsequent in
# If the directory doesn't exist, Docker creates it automatically
WORKDIR /app

# COPY copies files from your machine (build context) into the image
# Here we copy ONLY package.json first (not the whole app) to leverage Docker's
# if package.json hasn't changed, Docker reuses the cached "npm install" layer
COPY package.json .

# RUN executes a command DURING the image build (not at container runtime)
# This installs the app's dependencies as defined in package.json
RUN npm install

# Now copy the REST of the application code
# This is done AFTER npm install so that code changes don't invalidate the depe
COPY . .

# EXPOSE documents which port the app listens on INSIDE the container
# NOTE: this does NOT actually publish the port to the host - that's done with
EXPOSE 3000

# CMD defines the DEFAULT command that runs when a container starts from this i
# Only ONE CMD is allowed per Dockerfile (the last one wins if multiple are spe
# Written in "exec form" (JSON array) which is the recommended format
CMD ["node", "server.js"]
```

## Building and Running Your Image

```
bash
```

```
# Builds an image from the Dockerfile in the current directory  
# -t = tag (name) the image as "my-app:1.0"  
# "." = build context (the directory Docker sends to the daemon; it looks for "  
docker build -t my-app:1.0 .
```

```
# Runs a container from your custom image  
# -p 3000:3000 maps host port 3000 to the container's exposed port 3000  
docker run -d -p 3000:3000 --name my-app-container my-app:1.0
```

```
# Confirm it's running and check logs
```

```
docker ps
```

```
docker logs my-app-container
```

## Important Dockerfile Instructions Explained

| Instruction                | Purpose                                                                                                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>FROM</code>          | Sets the base image. Every Dockerfile must start with this.                                                                                                                                 |
| <code>WORKDIR</code>       | Sets/creates the working directory for following instructions.                                                                                                                              |
| <code>COPY</code>          | Copies files/folders from build context into the image.                                                                                                                                     |
| <code>ADD</code>           | Like <code>COPY</code> , but also supports auto-extracting tar files and remote URLs (generally, prefer <code>COPY</code> unless you need those extras).                                    |
| <code>RUN</code>           | Executes a command at <b>build time</b> , creates a new layer.                                                                                                                              |
| <code>CMD</code>           | Default command executed at <b>container start</b> . Can be overridden by <code>docker run &lt;image&gt; &lt;other-command&gt;</code> .                                                     |
| <code>ENTRYPOINT</code>    | Similar to <code>CMD</code> , but harder to override — used when you want the container to always run as a specific executable. Often combined with <code>CMD</code> for default arguments. |
| <code>EXPOSE</code>        | Documentation only — states which port the app uses.                                                                                                                                        |
| <code>ENV</code>           | Sets environment variables inside the image.                                                                                                                                                |
| <code>ARG</code>           | Defines build-time-only variables (not available in the running container).                                                                                                                 |
| <code>USER</code>          | Sets which user the following instructions/container run as (avoid running as root for security).                                                                                           |
| <code>.dockerignore</code> | A file (like <code>.gitignore</code> ) listing files/folders to exclude from the build context — speeds up builds and avoids leaking secrets.                                               |

## `.dockerignore` example

```
node_modules
.git
.env
*.log
```

## A Note on Layer Caching (Practical Tip)

Docker caches each instruction's result as a layer. If you change a line in your Dockerfile, **every instruction after that line is rebuilt** (cache invalidated), but everything before it is reused. That's why we copy `package.json` and run `npm install` *before* copying the rest of the source code — source code changes far more often than dependencies, so this ordering maximizes cache hits and speeds up builds dramatically.



---

## STEP 4: Docker Volumes — Persisting Data

### The Problem

Containers are **ephemeral** — when you delete a container, any data written inside its writable layer is **gone forever**. For databases, uploaded files, or any state you care about, you need persistence.

### Types of Data Storage in Docker

1. **Volumes** (recommended) — managed entirely by Docker, stored in a part of the host filesystem (`/var/lib/docker/volumes/` on Linux) that Docker controls.
2. **Bind mounts** — you map a specific folder on your host machine directly into the container. Full control, but less portable (depends on host's file structure).
3. **tmpfs mounts** — stored only in the host's memory (RAM), never written to disk. Used for sensitive, temporary data.

### Volume Commands

```
bash

# Creates a new named volume, managed by Docker
docker volume create my-data

# Lists all volumes on the system
docker volume ls

# Shows detailed info about a volume (where it lives on disk, driver, etc.)
docker volume inspect my-data

# Removes a specific volume (must not be in use by a running container)
docker volume rm my-data

# Removes ALL unused volumes - frees disk space, use carefully!
docker volume prune
```

### Using a Volume with a Container

```
bash
```

```
# -v <volume-name>:<container-path>
# This mounts the "my-data" volume to /var/lib/mysql INSIDE the container
# Even if the container is deleted, the data in "my-data" survives
docker run -d \
  --name my-database \
  -e MYSQL_ROOT_PASSWORD=secret \
  -v my-data:/var/lib/mysql \
  mysql:8.0
```

## Bind Mount Example (Great for Local Development)

```
bash
```

```
# Mounts a folder from your HOST machine directly into the container
# $(pwd)/src = absolute path to your local "src" folder (host side)
# /app/src = where it appears inside the container
# Changes on your host are IMMEDIATELY reflected inside the container - great f
docker run -d \
  -v $(pwd)/src:/app/src \
  --name dev-container \
  my-app:1.0
```

## Volumes vs Bind Mounts — When to Use Which

| Use Case                                         | Choice                                                    |
|--------------------------------------------------|-----------------------------------------------------------|
| Database storage in production                   | <b>Volume</b> (Docker-managed, portable, backup-friendly) |
| Local development with live code reload          | <b>Bind mount</b> (direct link to your source folder)     |
| Sharing config files from host                   | <b>Bind mount</b>                                         |
| Data that should be portable across environments | <b>Volume</b>                                             |

## STEP 5: Docker Networks — Connecting Containers

### The Problem

By default, containers are isolated. If you have a web app container and a database container, they need a way to talk to each other **without exposing the database to the outside world**.

## Docker Network Drivers (Theory)

| Driver  | Purpose                                                                                                                                                               |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bridge  | Default driver. Creates a private internal network on a single host. Containers on the same bridge network can communicate using container <b>names</b> as hostnames. |
| host    | Removes network isolation — container shares the host's network stack directly (no port mapping needed, but less isolation).                                          |
| none    | Completely disables networking for the container.                                                                                                                     |
| overlay | Connects containers across <b>multiple Docker hosts</b> (used in Docker Swarm for multi-machine clusters).                                                            |
| macvlan | Assigns a real MAC address to a container, making it appear as a physical device on the network.                                                                      |

## Network Commands

```
bash

# Lists all networks on this Docker host
docker network ls

# Creates a new custom bridge network (better than the default bridge - support
docker network create my-network

# Shows detailed info: connected containers, subnet, gateway, etc.
docker network inspect my-network

# Removes a network (must have no containers attached)
docker network rm my-network
```

## Connecting Containers via a Custom Network

```
bash
```

```
# Step 1: create a dedicated network for our app
```

```
docker network create app-network
```

```
# Step 2: run the database container ON that network
```

```
docker run -d \  
  --name db \  
  --network app-network \  
  -e POSTGRES_PASSWORD=secret \  
  postgres:15
```

```
# Step 3: run the app container on the SAME network
```

```
# Inside the app's code, it can now connect to the database using the hostname
```

```
# (Docker's built-in DNS resolves container NAMES to their internal IPs automat
```

```
docker run -d \  
  --name app \  
  --network app-network \  
  -e DATABASE_HOST=db \  
  my-app:1.0
```

**Key insight:** On a custom bridge network, Docker provides automatic DNS resolution — containers can reach each other by **container name** instead of needing to know IP addresses (which change anyway). This is the foundation that makes Docker Compose's multi-container setups "just work."

---

## STEP 6: Docker Compose — Managing Multi-Container Apps

### The Problem

Running multiple `docker run` commands manually (for app + database + cache + etc.) is tedious and error-prone. **Docker Compose** lets you define your entire multi-container application in **one YAML file** and spin it up with **one command**.

**Example:** `docker-compose.yml`

yaml

```
# The Compose file format version (v3.8 is widely supported)
```

```
version: "3.8"
```

```
# "services" defines each container that makes up your application
```

```
services:
```

```
# Service #1: our web application
```

```
app:
```

```
  build: .          # Build from the Dockerfile in the current directory
```

```
  ports:
```

```
    - "3000:3000"    # host_port:container_port
```

```
  environment:
```

```
    - DATABASE_HOST=db # Environment variable passed into the container
```

```
    - DATABASE_PORT=5432
```

```
  depends_on:
```

```
    - db              # Ensures "db" container starts BEFORE "app" (note: d
```

```
  volumes:
```

```
    - ./src:/app/src  # Bind mount for live code reload during development
```

```
  networks:
```

```
    - app-network
```

```
# Service #2: PostgreSQL database
```

```
db:
```

```
  image: postgres:15 # Pull this image directly instead of building
```

```
  environment:
```

```
    - POSTGRES_PASSWORD=secret
```

```
    - POSTGRES_DB=mydb
```

```
  volumes:
```

```
    - db-data:/var/lib/postgresql/data # Named volume for persistent DB stor
```

```
  networks:
```

```
    - app-network
```

```
# Service #3: Redis cache
```

```
cache:
```

```
  image: redis:7-alpine
```

```
  networks:
```

```
    - app-network
```

```
# Named volumes declared here so Compose knows to manage them
```

```
volumes:
```

```
  db-data:
```

```
# Custom network so all services can resolve each other by service name
```

```
networks:
```

```
  app-network:
```

```
app-network:  
  driver: bridge
```

## Compose Commands

```
bash  
  
# Builds images (if needed) and starts ALL services defined in docker-compose.y  
# -d = detached mode (runs in background)  
docker compose up -d  
  
# Lists the status of all services in this Compose project  
docker compose ps  
  
# Streams logs from ALL services (or specify a service name to filter: docker c  
docker compose logs -f  
  
# Stops and REMOVES all containers, networks created by "up" (but keeps volumes  
docker compose down  
  
# Same as above, but ALSO removes named volumes - deletes your data too, use ca  
docker compose down -v  
  
# Rebuilds images (useful after changing a Dockerfile or dependencies)  
docker compose build  
  
# Scales a specific service to run multiple instances (e.g., 3 copies of "app")  
docker compose up -d --scale app=3  
  
# Runs a one-off command inside a service's container (e.g., open a shell)  
docker compose exec app sh
```

## Why Compose Matters

- **Reproducibility:** Anyone can clone your repo and run `docker compose up` to get the exact same multi-service environment.
- **Service discovery for free:** Services reference each other by their service name (e.g., `db`, `cache`) thanks to Compose's automatic networking.
- **Single source of truth:** The whole app's architecture is documented in one readable YAML file.

---

## STEP 7: Docker Swarm — Orchestration at Scale

### The Problem

Compose is great for running multi-container apps on **one machine**. But what if you need to run your app across **multiple servers**, with automatic failover, load balancing, and zero-downtime deployments? That's **container orchestration**, and **Docker Swarm** is Docker's built-in orchestrator (an alternative to Kubernetes, simpler but less feature-rich).

## Swarm Theory

| Concept         | Meaning                                                                                                                                                                        |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Node            | A machine (physical or virtual) participating in the Swarm. Can be a <b>Manager</b> or <b>Worker</b> .                                                                         |
| Manager node    | Handles orchestration: scheduling tasks, maintaining cluster state, handling the API. Managers can also run workloads.                                                         |
| Worker node     | Only runs containers (tasks) assigned by managers; doesn't participate in cluster management decisions.                                                                        |
| Service         | A definition of a task to run across the Swarm (e.g., "run 5 replicas of my-app image"). Swarm's version of a "managed container."                                             |
| Task            | A single running container that's part of a service — Swarm schedules tasks onto nodes.                                                                                        |
| Stack           | A group of services deployed together, defined using a Compose-like YAML file ( <code>(docker stack deploy)</code> ).                                                          |
| Overlay network | A virtual network spanning multiple physical hosts, letting containers on different machines communicate as if on the same LAN.                                                |
| Routing mesh    | Swarm's built-in load balancer — any node in the cluster can receive a request on a published port and route it to a container running that service, even on a different node. |

## Setting Up a Swarm

```
bash
```

```
# On the FIRST machine, initialize the swarm - this machine becomes the first M  
# --advertise-addr = the IP address other nodes will use to reach this manager  
docker swarm init --advertise-addr 192.168.1.10
```

```
# This command outputs a "docker swarm join" command with a TOKEN  
# You run that exact output command on OTHER machines to add them as WORKERS, e  
docker swarm join --token SWMTKN-1-xxxxxxxxxxxxx 192.168.1.10:2377
```

```
# To add a MANAGER (not worker) node, generate the manager join token on an exi  
docker swarm join-token manager
```

```
# Lists all nodes in the swarm and their status (Leader, Manager, Worker, Ready  
docker node ls
```

## Deploying a Service to the Swarm

```
bash
```

```
# Creates a new service called "web" from the nginx image  
# --replicas 3 = run 3 identical copies (tasks), automatically load-balanced an  
# -p 8080:80 = publish port using the routing mesh (accessible on ANY node's IP  
docker service create --name web --replicas 3 -p 8080:80 nginx:latest
```

```
# Lists all services running in the swarm  
docker service ls
```

```
# Shows WHICH nodes each replica (task) of a service is running on  
docker service ps web
```

```
# Scales a service up or down dynamically  
docker service scale web=5
```

```
# Performs a ROLLING UPDATE - updates containers one at a time to avoid downtim  
docker service update --image nginx:1.25 web
```

```
# Removes a service entirely  
docker service rm web
```

## Deploying a Full Stack (Compose file + Swarm)

Swarm can consume a Compose-formatted YAML file directly:



```
bash
```

```
# Deploys an entire multi-service stack to the swarm using a Compose-style file
# "mystack" is the name given to this group of services
docker stack deploy -c docker-compose.yml mystack

# Lists all deployed stacks
docker stack ls

# Lists services belonging to a specific stack
docker stack services mystack

# Removes the entire stack and all its services
docker stack rm mystack
```

## Why Swarm (vs just Compose)

- **High availability:** if a node dies, Swarm automatically reschedules its containers onto healthy nodes.
- **Load balancing:** the routing mesh distributes traffic across replicas automatically.
- **Rolling updates & rollbacks:** update your app with zero downtime, and roll back instantly if something breaks.
- **Declarative desired state:** you say "I want 5 replicas of this service," and Swarm continuously works to keep that true.

*(Note: In the real world, Kubernetes has become the dominant orchestrator for large, complex deployments, while Swarm remains valued for its simplicity on smaller clusters.)*

---

## STEP 8: Common Beginner Mistakes

1. **Running everything as root inside containers** — Always create and use a non-root `USER` in your Dockerfile for security; a compromised root container is far more dangerous.
2. **Forgetting `.dockerignore`** — Without it, `COPY . .` sends your entire folder (including `node_modules`, `.git`, secrets) into the build context, causing slow builds and bloated/leaky images.
3. **Not leveraging layer caching** — Copying all files before installing dependencies invalidates the cache on every code change, making rebuilds painfully slow.
4. **Using `latest` tag in production** — `nginx:latest` today might not be the same image tomorrow. Always pin specific versions (`nginx:1.25.3`) for reproducible deployments.
5. **Storing data inside the container instead of a volume** — Then wondering why all their data vanished after `docker rm`.
6. **Not cleaning up unused resources** — Dangling images, stopped containers, and unused volumes silently eat disk space. Use `docker system prune` periodically.
7. **Confusing `EXPOSE` with actually publishing a port** — `EXPOSE` in a Dockerfile is just documentation; you still need `-p` (or `ports:` in Compose) to actually make the port reachable from the host.
8. **Putting secrets (passwords, API keys) directly in the Dockerfile with `ENV`** — These get baked into the image layers and are visible to anyone who can pull/inspect the image. Use environment variables at runtime, secrets managers, or Docker secrets instead.
9. **One giant container doing everything** — e.g., running app + database + cache all in one container. Docker's philosophy favors **one process per container** for better isolation, scaling, and maintainability.
10. **Not understanding `CMD` vs `ENTRYPOINT`** — Leading to confusion when trying to override container startup behavior at runtime.
11. **Ignoring image size** — Using a full `ubuntu` base image when a slim `alpine` variant would work fine, resulting in unnecessarily large, slow-to-pull images.
12. **Thinking `docker-compose down` deletes volumes by default** — It doesn't (you need the `-v` flag), which sometimes confuses people the *other* direction too (they expect data to persist but forget it's still there from an old volume).

---

## STEP 9: Interview Questions (Intermediate → Advanced)

### Q1: What is the difference between an image and a container?

An image is a read-only, immutable template/blueprint containing application code and dependencies. A container is a running (or stopped) instance created from that image, with its own writable layer on top. You can create many containers from a single image, just like creating many objects from one class.

## Q2: Explain Docker's layered filesystem and why it matters.

Each instruction in a Dockerfile creates a new, cached, read-only layer. Layers are stacked using a union filesystem (e.g., OverlayFS). When a container runs, Docker adds a thin writable layer on top. This design enables layer reuse/caching across images (saving disk space and speeding up builds) and means only the writable top layer changes at runtime.

## Q3: What's the difference between `CMD` and `ENTRYPOINT`?

`CMD` provides default arguments/command that can be **fully overridden** by arguments passed to `docker run`. `ENTRYPOINT` defines the command that **always runs**, and any `docker run` arguments are appended to it rather than replacing it. They're often combined: `ENTRYPOINT` sets the fixed executable, `CMD` supplies default arguments that a user can still override.

## Q4: How does a container differ fundamentally from a virtual machine?

Containers share the host OS kernel and isolate processes using Linux namespaces and cgroups, making them lightweight and fast to start. VMs virtualize entire hardware and run a full separate OS (with its own kernel) via a hypervisor, making them heavier and slower but offering stronger isolation.

## Q5: What happens to data written inside a container if the container is deleted, and how do you prevent data loss?

Data in the container's writable layer is destroyed when the container is removed. To persist data, use Docker **volumes** (Docker-managed storage) or **bind mounts** (mapped host directories), both of which exist independently of the container's lifecycle.

## Q6: Explain the difference between the default bridge network and a user-defined (custom) bridge network.

The default bridge network does NOT provide automatic DNS resolution between containers by name — you'd need to link containers manually or use IP addresses. A user-defined bridge network provides built-in DNS resolution, so containers can reach each other simply using their container/service name, plus better isolation from other unrelated containers.

## Q7: What is the difference between `docker stop` and `docker kill`?

`docker stop` sends a `SIGTERM` signal first (allowing the app to shut down gracefully), waits a grace period (default 10s), then sends `SIGKILL` if it hasn't exited. `docker kill` sends `SIGKILL` immediately, forcefully terminating the process with no cleanup time.

## Q8: What is a multi-stage build and why would you use one?

A multi-stage build uses multiple `FROM` statements in one Dockerfile, where later stages can selectively copy artifacts from earlier stages using `COPY --from=<stage>`. This lets you use a full SDK/build-tooling image to compile your app, then copy only the compiled output into a minimal final runtime image — dramatically reducing final image size by excluding build tools and intermediate files.

```
dockerfile

# Example multi-stage build
FROM golang:1.21 AS builder
WORKDIR /src
COPY . .
RUN go build -o app .

# Final minimal image - doesn't contain the Go compiler/toolchain, just the com
FROM alpine:latest
COPY --from=builder /src/app /app
CMD ["/app"]
```

### Q9: How does Docker Swarm's routing mesh work?

When a service publishes a port, every node in the swarm listens on that port, regardless of whether a task for that service is actually running on that particular node. Incoming requests to any node on that port are transparently routed (via IPVS-based load balancing) to a healthy container running the service, possibly on a different node.

### Q10: What's the difference between Docker Compose and Docker Swarm?

Compose orchestrates multiple containers as a single application **on one Docker host**, primarily for local development and simple deployments. Swarm extends this to **orchestrate services across a cluster of multiple hosts**, adding features like automatic failover, rolling updates, scaling, and load balancing across nodes. Notably, Swarm can consume Compose-formatted YAML files via `docker stack deploy`.

### Q11: What are Docker secrets, and why not just use environment variables for sensitive data?

Docker secrets (available in Swarm mode) are encrypted at rest, transmitted over encrypted TLS connections to the nodes that need them, and mounted as in-memory files inside the container — never exposed via `docker inspect` or image layers. Environment variables, by contrast, can leak via logs, `docker inspect`, process listings, or child processes, making them less safe for highly sensitive values like passwords or API keys.

### Q12: Explain the difference between `COPY` and `ADD`.

Both copy files into the image. `ADD` has extra features: it can automatically extract local tar archives into the destination, and it can fetch files from remote URLs. Because these implicit behaviors can be surprising/unsafe, best practice is to prefer `COPY` for straightforward file copying and only use `ADD` when you specifically need its extraction/URL features.

### Q13: How would you reduce the size of a Docker image?

Use minimal base images (e.g., `alpine` variants), leverage multi-stage builds to exclude build tools from the final image, combine `RUN` commands with `&&` to reduce layer count, remove package manager caches within the same `RUN` instruction (so they don't persist in a layer), use `.dockerignore` to avoid copying unnecessary files, and avoid installing unnecessary debugging tools in production images.

### Q14: What is the difference between `docker exec` and `docker attach`?

`docker exec` starts a **new process** inside an already-running container (e.g., opening a new shell session) without affecting the container's main process. `docker attach` connects your terminal directly to the container's **existing main process** (PID 1) — if you exit incorrectly (e.g., Ctrl+C instead of detaching with `Ctrl+P Ctrl+Q`), you could stop that main process, killing the container.

### Q15: How does health checking work in Docker, and why does it matter for orchestration?

A `HEALTHCHECK` instruction in a Dockerfile (or `healthcheck:` in Compose) defines a command Docker periodically runs inside the container to determine if the app is actually functioning correctly, not just that the process is alive. Orchestrators like Swarm use this status to decide whether to route traffic to a container or restart/replace it, enabling more reliable self-healing deployments.

```
dockerfile
```

```
# Example healthcheck: checks every 30s if the app responds on port 3000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1
```

### Q16: What's the difference between horizontal and vertical scaling in the context of Docker containers?

Vertical scaling means giving a single container more resources (CPU/memory limits raised). Horizontal scaling means running more replicas/instances of the same container (e.g., via `docker service scale` or `docker compose up --scale`), distributing load across them. Container orchestration tools like Swarm and Kubernetes are generally optimized around horizontal scaling since it also improves fault tolerance.

### Q17: What happens during `docker build` regarding the "build context"?

The build context is the set of files (typically the directory you specify, e.g., `(.)`) sent to the Docker daemon before the build starts. The daemon can only `COPY`/`ADD` files that are part of this context. A large or unfiltered build context (without `.dockerignore`) slows down builds significantly because that entire directory gets uploaded/streamed to the daemon.

---

## Quick Reference: The Full Command Progression

```
Images:      docker pull, docker build, docker images, docker rmi
Containers:  docker run, docker ps, docker stop/start, docker logs, docker
exec, docker rm
Volumes:     docker volume create/ls/inspect/rm
Networks:    docker network create/ls/inspect/rm
Compose:     docker compose up/down/logs/build/scale
Swarm:       docker swarm init/join, docker service create/scale/update,
docker stack deploy
```

You've now gone from "**what is a container**" all the way to **orchestrating services across a multi-node cluster**. The natural next step beyond this guide (not covered here) is **Kubernetes**, which builds on the same core concepts (images, volumes, networks) but with a much larger and more powerful orchestration feature set.